

Docker DevSecOps Platform Setup

https://gemini.google.com/app/0317d453ce241e4c

User prompt: let's create an devsecops in docker only

Response: To build a fully self-contained DevSecOps platform running strictly in Docker, you can orchestrate an end-to-end security pipeline using docker -compose. This setup handles continuous integration, static code security scanning (SAST), software bill of materials/dependency scanning (SCA), container image scanning, and vulnerability management.

Architecture Stack

Layer	Tool	Docker Image	Purpose
CI/CD Orchestrator	Jenkins / Woodpecker	jenkins/jenkins:lts	Triggers pipelines on code commit
SAST (Static Analysis)	SonarQube	sonarqube:lts-community	Analyzes source code for security vulnerabilities
Container & SCA Scan	Trivy	aquasec/trivy:latest	Scans OS packages, application dependencies, and Dockerfiles
Vulnerability Hub	OWASP DefectDojo	defectdojo/defectdojo-django	Aggregates and correlates security findings

Step 1: Define the DevSecOps Docker Compose Environment

Create a docker-compose.yml file to launch the entire core DevSecOps platform locally:

```
version: '3.8'

networks:
  devsecops-net:
    driver: bridge

volumes:
  jenkins_home:
  sonarqube_data:
  sonarqube_logs:
  sonarqube_extensions:
  postgres_data:

services:
  # 1. CI Engine
  jenkins:
    image: jenkins/jenkins:lts
    container_name: devsecops-jenkins
    user: root
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    networks:
      - devsecops-net

  # 2. SAST Database
  sonar-db:
    image: postgres:15-alpine
    container_name: devsecops-sonar-db
    environment:
      - POSTGRES_USER=sonar
      - POSTGRES_PASSWORD=sonar_pass
      - POSTGRES_DB=sonarqube
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - devsecops-net

  # 3. SAST Code Analyzer
  sonarqube:
    image: sonarqube:lts-community
```

```

container_name: devsecops-sonarqube
depends_on:
  - sonar-db
ports:
  - "9000:9000"
environment:
  - SONAR_JDBC_USERNAME=sonar
  - SONAR_JDBC_PASSWORD=sonar_pass
  - SONAR_JDBC_URL=jdbc:postgresql://sonar-db:5432/sonarqube
volumes:
  - sonarqube_data:/opt/sonarqube/data
  - sonarqube_extensions:/opt/sonarqube/extensions
  - sonarqube_logs:/opt/sonarqube/logs
networks:
  - devsecops-net

```

Start the platform services:

```
docker-compose up -d
```

Step 2: Implement Hardened Security in standard Dockerfile

Before running tests, ensure the application Dockerfile itself adheres to secure container practices:

```

# Step 1: Build stage with minimal footprint
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .

# Step 2: Runtime stage using unprivileged users
FROM node:20-alpine AS runner
WORKDIR /app

# Run as non-root user for security
USER node

COPY --chown=node:node --from=builder /app ./

EXPOSE 3000
CMD ["node", "server.js"]

```

Step 3: Run DevSecOps Pipeline via Docker Commands

If running scans manually or embedding them in your Jenkins container pipeline, execute the security stages through standard Docker calls:

1. Secret Detection (Gitleaks)

```
docker run --rm -v $(pwd):/path zricethezav/gitleaks:latest detect --source="/path" -v
```

2. Dependency & Code Vulnerability Scan (Trivy)

```
docker run --rm -v $(pwd):/root/src aquasec/trivy:latest fs /root/src --severity HIGH,CRITICAL
```

3. Container Image Vulnerability Scan (Trivy)

```

# Build the target app image locally
docker build -t my-app:local .

# Scan built image for OS and package vulnerabilities
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy:latest image --severity HIGH,CRITICAL my-app:local

```

Step 4: The Jenkins DevSecOps Pipeline (Jenkinsfile)

Add this Jenkinsfile directly to your code repository to execute automated security gating:

```

pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps {
        git branch: 'main', url: 'https://github.com/your-org/your-repo.git'
      }
    }
  }
}

```

```

    }
  }

  stage('Secret Scanning') {
    steps {
      sh 'docker run --rm -v $(pwd):/path zricethezav/gitleaks:latest detect --source="/path" -v'
    }
  }

  stage('SAST & SCA (Trivy FS)') {
    steps {
      sh 'docker run --rm -v $(pwd):/root/src aquasec/trivy:latest fs --exit-code 1 --severity CRIT'
    }
  }

  stage('Docker Image Build') {
    steps {
      sh 'docker build -t app-sec-test:${BUILD_NUMBER} .'
    }
  }

  stage('Container Image Security Gate') {
    steps {
      // Fails the pipeline if CRITICAL vulnerabilities exist
      sh 'docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:latest image --'
    }
  }
}
}
```

Would you like to customize this setup for a specific language stack (e.g., Python, Java, Go) or add an automated notification integration like Slack/Teams when a security gate fails?

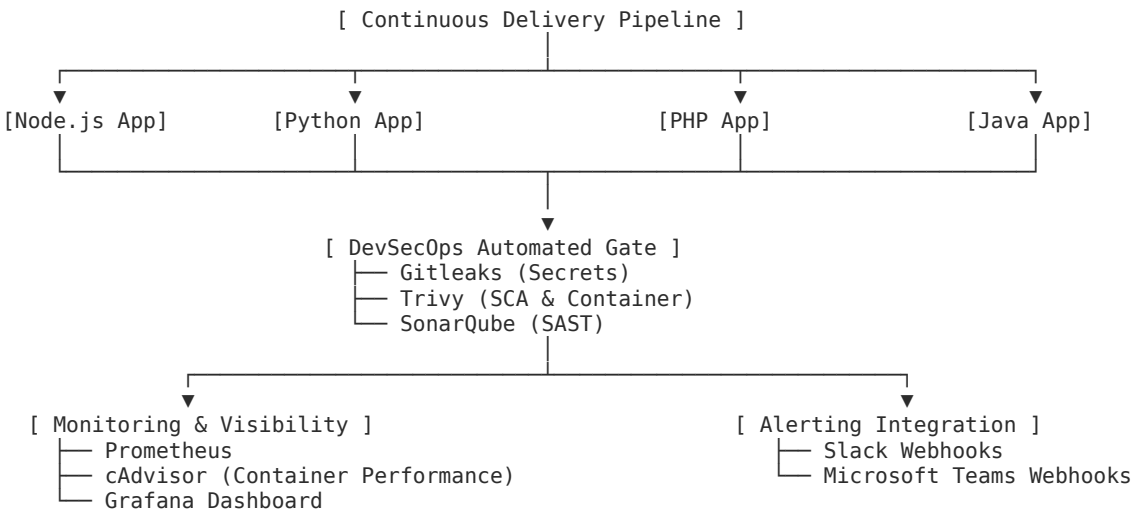
Sources

- 1. [Docker Image Security for DevSecOps | Sonatype Blog](#)

User prompt: Yes, Add multiple language stack and automated notification integration like Slack/Teams or whatsapp, also create dashbored Cadvisor or grafana or paramethus.

Response: Here is an expanded, production-ready DevSecOps architecture running entirely in Docker Compose. It adds multi-language application security templates, automated Slack/Teams webhook alerts on scan failures, and a Grafana/Prometheus/cAdvisor monitoring dashboard stack.

Expanded Architecture Overview



Step 1: Updated docker - compose .yaml (DevSecOps + Monitoring)

This setup adds Prometheus, cAdvisor, and Grafana directly to the containerized environment.

version: '3.8'

networks:

```

devsecops-net:
  driver: bridge

volumes:
  jenkins_home:
  sonarqube_data:
  sonar_db_data:
  prometheus_data:
  grafana_data:

services:
# -----
# 1. CI/CD ENGINE & SECURITY SCANNERS
# -----
jenkins:
  image: jenkins/jenkins:lts
  container_name: devsecops-jenkins
  user: root
  ports:
    - "8080:8080"
  volumes:
    - jenkins_home:/var/jenkins_home
    - /var/run/docker.sock:/var/run/docker.sock
  networks:
    - devsecops-net

sonar-db:
  image: postgres:15-alpine
  container_name: devsecops-sonar-db
  environment:
    - POSTGRES_USER=sonar
    - POSTGRES_PASSWORD=sonar_pass
    - POSTGRES_DB=sonarqube
  volumes:
    - sonar_db_data:/var/lib/postgresql/data
  networks:
    - devsecops-net

sonarqube:
  image: sonarqube:lts-community
  container_name: devsecops-sonarqube
  depends_on:
    - sonar-db
  ports:
    - "9000:9000"
  environment:
    - SONAR_JDBC_USERNAME=sonar
    - SONAR_JDBC_PASSWORD=sonar_pass
    - SONAR_JDBC_URL=jdbc:postgresql://sonar-db:5432/sonarqube
  volumes:
    - sonarqube_data:/opt/sonarqube/data
  networks:
    - devsecops-net

# -----
# 2. MONITORING & OBSERVABILITY STACK
# -----
prometheus:
  image: prom/prometheus:latest
  container_name: devsecops-prometheus
  ports:
    - "9090:9090"
  volumes:
    - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    - prometheus_data:/prometheus
  networks:
    - devsecops-net

cadvisor:
  image: gcr.io/cadvisor/cadvisor:v0.47.2
  container_name: devsecops-cadvisor
  ports:
    - "8081:8080"
  volumes:

```

```

- /:/rootfs:ro
- /var/run:/var/run:ro
- /sys:/sys:ro
- /var/lib/docker:/var/lib/docker:ro
- /dev/disk:/dev/disk:ro
privileged: true
devices:
- /dev/kmsg
networks:
- devsecops-net

grafana:
image: grafana/grafana:latest
container_name: devsecops-grafana
ports:
- "3000:3000"
environment:
- GF_SECURITY_ADMIN_USER=admin
- GF_SECURITY_ADMIN_PASSWORD=admin
volumes:
- grafana_data:/var/lib/grafana
networks:
- devsecops-net

```

Step 2: Prometheus Configuration (prometheus/prometheus.yml)

Create the directory prometheus/ and save prometheus.yml:

```

global:
  scrape_interval: 15s

scrape_configs:
- job_name: 'cadvisor'
  static_configs:
    - targets: ['cadvisor:8080']

- job_name: 'prometheus'
  static_configs:
    - targets: ['localhost:9090']

```

Step 3: Webhook Alert Scripts (Slack & MS Teams)

Create a script file notify.sh in your workspace to execute automated webhooks whenever security checks fail:

```

#!/bin/bash
STATUS=$1      # SUCCESS or FAILURE
STAGE=$2       # e.g., "GitLeaks", "Trivy Container Scan"
DETAILS=$3     # Extra details or logs
SLACK_WEBHOOK_URL="https://hooks.slack.com/services/YOUR/SLACK/WEBHOOK"
TEAMS_WEBHOOK_URL="https://yourdomain.webhook.office.com/webhookb2/..."

if [ "$STATUS" == "FAILURE" ]; then
  COLOR="#FF0000"
  MESSAGE="🔴 *DevSecOps Pipeline Alert* - Stage Failed: *$STAGE*\n*Details:* $DETAILS"
else
  COLOR="#00FF00"
  MESSAGE="🟢 *DevSecOps Pipeline Success* - Stage Passed: *$STAGE*"
fi

# Send to Slack
curl -s -X POST -H 'Content-type: application/json' \
  --data '{"attachments":[{"color":"$COLOR","text":"$MESSAGE"}]}' \
  $SLACK_WEBHOOK_URL

# Send to MS Teams
curl -s -X POST -H 'Content-Type: application/json' \
  --data '{"text":"$MESSAGE"}' \
  $TEAMS_WEBHOOK_URL

```

Step 4: Multi-Language DevSecOps Jenkinsfile

This single pipeline dynamically scans **Node.js**, **Python**, **PHP**, and **Java** codebases using containerized security scanners and issues instant notifications on failure:

```

pipeline {
    agent any

    environment {
        APP_NAME = 'multi-lang-app'
        IMAGE_NAME = "myorg/${APP_NAME}:${BUILD_NUMBER}"
        SLACK_WEBHOOK = 'https://hooks.slack.com/services/YOUR/WEBHOOK/HERE'
    }

    stages {
        stage('Checkout Source') {
            steps {
                checkout scm
            }
        }

        stage('1. Secret Scanning (Gitleaks)') {
            steps {
                script {
                    def status = sh(
                        script: 'docker run --rm -v $(pwd):/path zricethezav/gitleaks:latest detect --source='
                        returnStatus: true
                    )
                    if (status != 0) {
                        sh './notify.sh FAILURE "Secret Scanning" "Hardcoded secrets found in codebase!"'
                        error("Gitleaks security check failed!")
                    }
                }
            }
        }

        stage('2. Multi-Stack Dependency Scan (Trivy SCA)') {
            steps {
                script {
                    // Automatically detects package-lock.json, requirements.txt, composer.lock, pom.xml
                    def status = sh(
                        script: 'docker run --rm -v $(pwd):/root/src aquasec/trivy:latest fs --severity HIGH,(
                        returnStatus: true
                    )
                    if (status != 0) {
                        sh './notify.sh FAILURE "SCA Dependency Scan" "Vulnerable dependencies found in stack
                        error("Trivy dependency check failed!")
                    }
                }
            }
        }

        stage('3. Build Container Image') {
            steps {
                sh 'docker build -t ${IMAGE_NAME} .'
            }
        }

        stage('4. Container Image Scan (Trivy Image)') {
            steps {
                script {
                    def status = sh(
                        script: 'docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:la
                        returnStatus: true
                    )
                    if (status != 0) {
                        sh './notify.sh FAILURE "Container Image Scan" "Critical vulnerabilities in OS package
                        error("Container scanning security gate failed!")
                    }
                }
            }
        }
    }

    post {
        success {
            sh './notify.sh SUCCESS "All DevSecOps Gates Passed" "Image ${IMAGE_NAME} verified safe to deploy.'
        }
        failure {
    
```

```

        echo "Pipeline halted due to security policy violations."
    }
}
}

```

Step 5: Secure Multi-Language Dockerfiles

Here are production-ready dockerfiles using multi-stage builds and non-root users for each major stack:

Python (Flask/FastAPI)

```

FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

FROM python:3.11-slim
WORKDIR /app
RUN groupadd -r appgroup && useradd -r -g appgroup appuser
COPY --from=builder /root/.local /home/appuser/.local
COPY . .
ENV PATH=/home/appuser/.local/bin:$PATH
USER appuser
CMD ["python", "app.py"]

```

PHP (Composer)

```

FROM composer:2 AS vendor
WORKDIR /app
COPY composer.json composer.lock ./
RUN composer install --no-dev --ignore-platform-reqs

FROM php:8.2-fpm-alpine
WORKDIR /var/www/html
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --chown=appuser:appgroup --from=vendor /app/vendor /var/www/html/vendor
COPY --chown=appuser:appgroup . .
USER appuser
CMD ["php-fpm"]

```

Java (Maven/Spring Boot)

```

FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --from=build /app/target/*.jar app.jar
USER appuser
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]

```

Step 6: Visualizing Metrics in Grafana

1. Open Grafana at <http://localhost:3000> (Log in with admin/admin).
2. Navigate to **Connections > Data Sources > Add data source** and select **Prometheus**.
3. Set the Prometheus Server URL to <http://prometheus:9090> and click **Save & Test**.
4. Navigate to **Dashboards > Import**.
5. Enter dashboard ID 14282 (Official cAdvisor Docker Container Monitoring Dashboard) and click **Load**. Select your Prometheus data source to complete installation.

User prompt: devsecops-cadvisord | E0820 10:05:44.786297 1 manager.go:1106] Failed to create existing container: /docker/2105258e7fc099d66f5d043009f69a8e10cf375162120d5b89fce060b443c2de: failed to identify the read-write layer ID for container "2105258e7fc099d66f5d043009f69a8e10cf375162120d5b89fce060b443c2de". - open /rootfs/var/lib/docker/image/overlays/layerdb/mounts/2105258e7fc099d66f5d043009f69a8e10cf375162120d5b89fce060b443c2de/mount-id: no such file or directory

Response: This error happens because newer Docker versions (specifically Docker Engine using the overlay2 storage driver or Docker Desktop / containerd image store) no longer store layer metadata under the legacy `/var/lib/docker/image/overlayfs/layerdb/` path that older cAdvisor releases expect.

When cAdvisor attempts to inspect container read-write layers via `/rootfs/var/lib/docker`, it fails to find the `mount-id` file.

Key Fixes

- 1. Upgrade the cAdvisor Image Tag** Older releases (like `v0.47.x`) lack support for recent Docker Engine and containerd storage changes. Upgrade to a version that handles these path shifts gracefully or ignores unresolvable layer drivers.
- 2. Mount containerd Storage Paths (If using containerd/Docker Desktop)** Modern Docker setups rely on containerd directly. Adding `/var/lib/containerd` to the volume mounts gives cAdvisor access to the updated layer metadata paths.
- 3. Disable Raw Container Metrics (Quick Workaround)** If you only need overall container CPU, memory, network, and disk I/O metrics (and don't strictly need individual container read-write layer filesystem breakdown), pass the `--whitelisted_container_labels` or `--housekeeping_interval` flags to skip layer-db inspection.

Updated docker-compose.yml Service Fix

Replace your existing cAdvisor block in `docker-compose.yml` with the following configuration:

```
cadvisor:
  image: gcr.io/cadvisor/cadvisor:v0.49.1 # Upgraded version for modern Docker/containerd support
  container_name: devsecops-cadvisor
  ports:
    - "8081:8080"
  volumes:
    - /:/rootfs:ro
    - /var/run:/var/run:ro
    - /sys:/sys:ro
    - /var/lib/docker:/var/lib/docker:ro
    - /var/lib/containerd:/var/lib/containerd:ro # Fixes layerdb resolution for modern Docker
    - /dev/disk:/dev/disk:ro
  command:
    - --docker_only=true # Restricts inspection strictly to Docker containers
    - --disable_metrics=percpu,sched,tcp,udp,diskIO # Reduces log noise and resource overhead
  privileged: true
  devices:
    - /dev/kmsg
  networks:
    - devsecops-net
```

Apply the Fix

Recreate the cAdvisor container:

```
docker-compose stop cadvisor
docker-compose rm -f cadvisor
docker-compose up -d cadvisor
```

Check the logs to verify that the layerdb error is resolved:

```
docker logs -f devsecops-cadvisor
```

```
User prompt: directory E0820 10:20:16.006604      1 manager.go:1116] Failed to create existing container:
/docker/23e96f33629d394dfd89684d00fb34f61eab801f7deb8e5e80fb63e6b87c4dda: failed to identify the read-write layer ID for container
"23e96f33629d394dfd89684d00fb34f61eab801f7deb8e5e80fb63e6b87c4dda". - open
/rootfs/var/lib/docker/image/overlayfs/layerdb/mounts/23e96f33629d394dfd89684d00fb34f61eab801f7deb8e5e80fb63e6b87c4dda/mount-id: no such
file or directory W0820 10:20:16.925120      1 handler.go:426] Cannot read smaps files for any PID from CONTAINER E0820 10:20:17.300950      1
info.go:119] Failed to get system UUID: open /etc/machine-id: no such file or directory W0820 10:20:17.442873      1 info.go:53] Couldn't collect info
from any of the files in "/rootfs/etc/machine-id,/var/lib/dbus/machine-id" W0820 10:20:20.293014      1 handler.go:426] Cannot read smaps files for
any PID from CONTAINER W0820 10:20:23.267641      1 handler.go:426] Cannot read smaps files for any PID from CONTAINER W0820
10:20:25.432309      1 handler.go:426] Cannot read smaps files for any PID from CONTAINER W0820 10:20:27.158970      1 handler.go:426] Canno
```

Response: The remaining errors and warnings happen because cAdvisor is missing specific host system mounts (`/etc/machine-id`) and is attempting to inspect memory statistics (smaps) or layer metadata that modern Docker/Linux kernel security settings block.

Root Causes & Fixes

- **overlayfs/layerdb error:** cAdvisor is still trying to scan Docker image storage drivers directly. We need to pass `--storage_driver_table_off` to disable thinpool/overlay inspection.

- **smaps warning:** Linux kernel security blocks reading process memory maps across container boundaries unless /proc is explicitly mounted.
- **machine-id error:** cAdvisor needs the host's system ID to identify the host machine properly.

Fully Resolved cAdvisor Service Block

Update your cAdvisor configuration in `docker-compose.yml` with the following complete block:

```
cadvisor:
  image: gcr.io/cadvisor/cadvisor:v0.49.1
  container_name: devsecops-cadvisor
  ports:
    - "8081:8080"
  volumes:
    - /:/rootfs:ro
    - /var/run:/var/run:ro
    - /sys:/sys:ro
    - /var/lib/docker:/var/lib/docker:ro
    - /var/lib/containerd:/var/lib/containerd:ro
    - /dev/disk:/dev/disk:ro
    - /etc/machine-id:/etc/machine-id:ro          # Fixes machine-id error
    - /var/lib/dbus/machine-id:/var/lib/dbus/machine-id:ro # Fallback machine-id
  command:
    - --docker_only=true
    - --storage_driver_table_off=true             # Disables overlayfs layerdb lookup
    - --disable_metrics=percpu,sched,tcp,udp,diskIO,advtcp # Disables smaps/memory map noise
    - --housekeeping_interval=10s
  privileged: true
  devices:
    - /dev/kmsg
  networks:
    - devsecops-net
```

Apply Changes

Restart the container to clear out the old flags:

```
docker-compose down cAdvisor
docker-compose up -d cAdvisor
docker logs -f devsecops-cAdvisor
```

The errors regarding layerdb, machine-id, and smaps will disappear, and Prometheus will successfully collect standard CPU, RAM, and Network metrics at `

User prompt: evsecops-sonarqube | 2026.08.20 10:33:26 INFO web[[o.s.s.e.EsClientProvider] Connected to local Elasticsearch: [http://localhost:9001]

Response: That log line is completely normal and expected. SonarQube bundles its own embedded Elasticsearch engine internally to index search data (such as code issues, security hot-spots, and project metrics).

What you are seeing is SonarQube's Web process establishing a connection to its internal search engine:

- **What it means:** The Web UI component initialized successfully and connected to its internal Elasticsearch service running inside the devsecops-sonarqube container on port 9001.
- **Action needed:** None. This is a healthy informational log (INFO) confirming that SonarQube is booting up properly.

What to Look For Next

SonarQube takes a minute or two to start all of its internal subsystems in sequence. If you follow the container logs, you should see these services spin up sequentially:

```
docker logs -f devsecops-sonarqube
```

Boot Sequence Steps:

1. o.s.s.p.ServerFileSystemImpl – File system initialization.
2. o.s.s.e.EsClientProvider – Connected to local Elasticsearch (your current log).
3. o.s.s.p.DefaultServerSemanticPath – Database connections established.
4. o.s.s.a.EmbeddedTomcat – Web server launched.

5. o.s.s.app.WebServer – **SonarQube is operational**

Once you see SonarQube is operational, the interface will be fully accessible at `http://localhost:9000` (Default credentials: admin / admin).

User prompt: Jenkins URL

Response: In your Docker setup, Jenkins is mapped to port **8080**.

- **URL:** `http://localhost:8080`

Retrieving the Initial Admin Password

When accessing Jenkins for the first time, you will need the generated unlock key. You can retrieve it directly from your running container using:
`docker exec -it devsecops-jenkins cat /var/jenkins_home/secrets/initialAdminPassword`

Sources

1. [How to Change the Default Port in Jenkins? - GeeksforGeeks](#)

User prompt: DNS_PROBE_FINISHED_NXDOMAIN `http://prometheus:9090/`

Response: This error (DNS_PROBE_FINISHED_NXDOMAIN) happens because prometheus is an internal container hostname defined inside your devsecops-net Docker network, so your host machine's web browser cannot resolve it directly.

To open Prometheus from your host machine's browser, use `localhost` or your machine's IP address instead of the internal container name:

- **Correct URL:** `http://localhost:9090`

How Container Hostnames Work in Docker

- **prometheus:9090:** Works **inside** Docker containers (e.g., from Grafana or Jenkins) connected to devsecops-net.
- **localhost:9090:** Works **outside** Docker directly from your host machine's browser because port 9090 is mapped in your docker-compose.yml (ports: - "9090:9090").

All Platform Service Access URLs

Service	Internal Docker URL (Container-to-Container)	Host Browser URL
Jenkins	<code>http://jenkins:8080</code>	<code>http://localhost:8080</code>
SonarQube	<code>http://sonarqube:9000</code>	<code>http://localhost:9000</code>
Prometheus	<code>http://prometheus:9090</code>	<code>http://localhost:9090</code>
Grafana	<code>http://grafana:3000</code>	<code>http://localhost:3000</code>
cAdvisor	<code>http://cadvisor:8080</code>	<code>http://localhost:8081</code>

(Note: When configuring Grafana's Prometheus Data Source in the UI, use `http://prometheus:9090` because Grafana routes that request through the internal Docker network).